

Technical Analysis - Identity Checker

1. Problem Statement

The project addresses identity verification from one image: a selfie where the person is holding a visible identity document. The goal is to verify whether the live face in the scene is compatible with the face printed on the document. This problem is relevant because document-based onboarding workflows often require a user to prove that the document belongs to the person currently present in front of the camera.

The project is an academic prototype. It is not intended for real identity control, legal verification, or production biometric decision-making.

2. Dataset

The expected dataset is synthetic and contains single images where a person is holding a document. Each sample is labeled as:

- ``match``: the live face and the document face represent the same identity.
- ``no_match``: the document face does not match the live face.

The evaluation CSV must contain ``image,label`` columns. Positive-only datasets can estimate recall/success rate but cannot fully estimate false positives, precision, F1-score, or a complete confusion matrix.

3. Pipeline

The application implements a complete computer vision pipeline:

1. Data acquisition: the Flask interface receives one uploaded image containing both the live face and the document.
2. Preprocessing: the image is decoded with OpenCV, all visible faces are detected, and face regions are cropped or aligned.
3. Face selection: detected faces are sorted by area. The largest face is treated as the live/selfie face; smaller faces are treated as candidate document portraits.
4. Feature representation: the deep pipeline uses SFace neural embeddings. The fallback pipeline uses grayscale intensity, Local Binary Patterns, and gradient histograms.
5. Core logic: the system compares the live-face embedding with each

document-face candidate, keeps the best cosine similarity, and applies a configurable threshold.

6. Post-processing: the best candidate is selected, scores are rounded, and an annotated preview highlights the live face and selected document face.

The project exposes both a browser interface and a JSON API at `/api/verify``, which makes the pipeline usable from scripts or external clients.

It also exposes `/health`` for service and model-file status checks during demo or deployment.

The architecture is documented in `docs/architecture.md``, while the API contract is documented in `docs/api.md`` and `openapi.json``.

4. Algorithms and Architecture

The primary method uses OpenCV Zoo models:

- YuNet for face detection.
- SFace for face recognition and embedding extraction.

Both models are loaded with OpenCV DNN through `opencv-contrib-python``.

The use of pre-trained models is justified because the project focuses on the end-to-end computer vision pipeline and identity-verification workflow, while the custom logic is in the single-image face selection, candidate comparison, thresholding, fallback path, evaluation, and web integration.

Model files can be prepared with `python scripts/download_models.py``. If they are missing at runtime, the app attempts to download them automatically before falling back to the classical pipeline.

The pre-trained model usage, intended scope, limitations, and ethical constraints are summarized in `docs/model_card.md``. External references are listed in `docs/references.md``.

If the ONNX models are unavailable, the system falls back to a classical method:

- Haar Cascade face detection.
- Histogram equalization.
- Local Binary Pattern texture descriptor.
- Gradient-orientation histogram.
- Cosine similarity and threshold decision.

5. Experimental Protocol

Run evaluation with:

```
```powershell
python evaluate.py samples.csv --threshold 0.363 --output
evaluation_results.json
```
```

The script reports:

- accuracy;
- precision;
- recall;
- F1-score;
- confusion matrix;
- confusion-matrix plot;
- score-distribution plot;
- row-level predictions and errors.

The recommended experiment is to create a balanced synthetic set with both `match` and `no\_match` samples. If only positive samples are available, report the number of successful verifications and clearly state that false-positive behavior is not measured. The repository also includes scripts to build the CSV from labeled folders and to tune the decision threshold on a validation set.

6. Results

This section must be completed after running the evaluator on the final dataset. Paste the generated metrics from `evaluation\_results.json` and, if possible, include a small table with true positives, true negatives, false positives, and false negatives.

Suggested table:

| Metric | Value |
|-----------|--------------|
| Accuracy | To be filled |
| Precision | To be filled |
| Recall | To be filled |
| F1-score | To be filled |

7. Failure Analysis

Expected failure cases include:

- the document portrait is too small or blurry;
- glare or plastic reflections hide the document face;
- the live face is partially occluded;
- strong pose differences between live face and document face;
- multiple unrelated faces appear in the scene;
- the largest detected face is not the live user;
- synthetic data is visually too different from real camera images.

These cases should be documented with examples from the dataset when available.

8. Ethical and Security Considerations

The system processes sensitive biometric and document data. For this reason:

- uploaded files are deleted after verification;
- upload size is limited;
- filenames are sanitized;
- the prototype should not store personal documents or selfies;
- the system should not be used for real identity decisions;
- biometric systems can show demographic bias and different performance across lighting, camera quality, age, skin tone, and document type;
- results should be interpreted as a computer vision experiment, not as proof of identity.

9. Reproducibility

The repository includes:

- Flask source code;
- JSON API endpoint;
- API and architecture documentation;
- OpenAPI specification;
- `requirements.txt`;
- evaluation script;
- generated plots for the experimental section;
- dataset CSV builder and threshold tuning scripts;
- model download/setup script;
- automated tests;
- project audit and submission checklist;

- privacy/ethics documentation;
- model card and references;
- GitHub Actions test workflow;
- Docker deployment files;
- sample CSV format;
- README with setup and running instructions;
- this technical analysis document.